Deep Learning

Do Neurônio Biológico ao Perceptron e ao MLP

Prof. Bruno Menezes | FAETEC Resende | Inteligência Artificial Aplicada

Roteiro de Hoje

01

O que é Inteligência Artificial?

Definição, contexto histórico e aplicações

02

Como máquinas aprendem?

Tipos de aprendizado: supervisionado, não supervisionado, reforço

03

O primeiro modelo — McCulloch & Pitts (1943)

Neurônio artificial e lógica binária

04

Perceptron — Rosenblatt (1957)

Pesos, bias e separação linear

05

O problema do XOR e o Inverno da IA

Limitações do Perceptron simples

06

MLP e Backpropagation — Hinton (1986)

Redes multicamada e aprendizado profundo

07

Funções de Ativação

Sigmoid, ReLU, Tanh — por que importam?

O que é Inteligência Artificial?

> Ciência e engenharia que produz máquinas e programas de computador inteligentes.
>
> *— John McCarthy, 2007 (criador do termo "Inteligência Artificial")*

🤔 Inteligência Natural

Capacidade de obter informações, aprender com a experiência, adaptar-se e usar o raciocínio (APA, 2023)

🤖 Inteligência Artificial

Reprodução computacional de comportamentos inteligentes para resolver tarefas específicas

🧠 Por que estudar?

IA já está presente em voz, imagem, texto, robótica, saúde, finanças e no seu cotidiano

Como Máquinas Aprendem?

🏫

Supervisionado

Como um aluno com gabarito

O modelo aprende com exemplos rotulados (entrada + resposta correta). Ex: classificar e-mails como spam ou não-spam.

Ex: Classificação, Regressão

🔍

Não Supervisionado

Como explorar um mapa desconhecido

O modelo descobre padrões nos dados sem respostas pré-definidas. Ex: agrupar clientes por comportamento de compra.

Ex: Clustering (K-Means), Redução dimensional

🎮

Por Reforço

Como aprender a jogar um videogame

Um agente aprende tentativa e erro, recebendo recompensas por ações corretas. Ex: AlphaGo, carros autônomos.

Ex: Jogos, Robótica, Navegação

⚖️

Semi-Supervisionado

Poucos exemplos rotulados + muitos não rotulados

Combina dados rotulados e não rotulados. Útil quando rotular dados é caro ou trabalhoso.

Ex: Reconhecimento de imagens médicas

A Jornada Histórica

Do neurônio biológico ao Deep Learning moderno

1943
McCulloch & Pitts

1957
Perceptron Rosenblatt

1969
Crise XOR Minsky

1986
MLP Backprop

2006+
Deep Learning

1943 — O Primeiro Neurônio Artificial

McCulloch & Pitts

Inspiração: O Neurônio Biológico

Dendritos → Recebem sinais de outros neurônios (entradas)

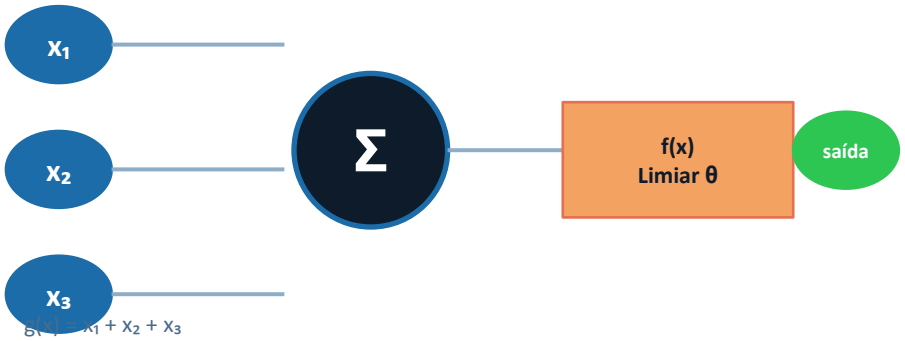
Corpo celular → Soma todos os sinais recebidos

Axônio → Transmite o sinal de saída

Sinapses → Conexões com outros neurônios (pesos)

Limiar → Só dispara se o sinal acumulado for forte o suficiente

O Modelo Matemático (1943)



Regra de ativação:

SE $g(x) \geq \theta$ → saída = 1 (neurônio DISPARA)

SE $g(x) < \theta$ → saída = 0 (neurônio SILENCIOSO)

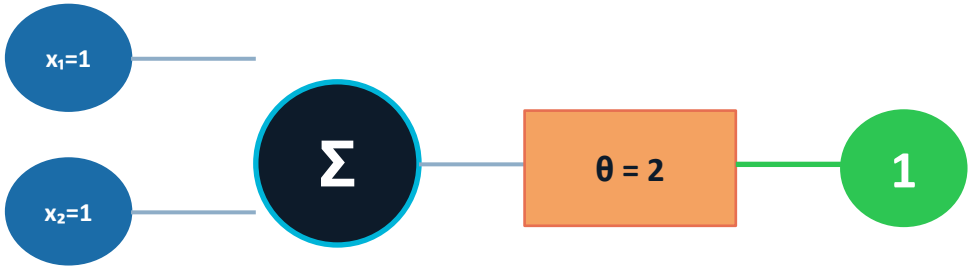
Limitação: apenas entradas binárias (0 ou 1), sem aprendizado — os pesos eram fixos!

McCulloch & Pitts — Exemplo: Porta AND

Tabela Verdade — AND

x_1	x_2	$g(x) = x_1+x_2$	$\theta = 2$	Saída
0	0	0	< 2	0
1	0	1	< 2	0
0	1	1	< 2	0
1	1	2	≥ 2	1 ✓

Como funciona o AND com θ = 2?



1 + 1 = 2 → 2 ≥ 2 → DISPARA → saída = 1

Conclusão:
O modelo de McCulloch & Pitts provava que neurônios podiam **realizar lógica formal** (AND, OR, NOT).

Porém: não aprendia. Os pesos e o limiar eram definidos manualmente pelo programador.

1957 — O Perceptron de Rosenblatt

O que mudou em relação a 1943?

Antes: Pesos fixos (manual)

Agora: Pesos ajustáveis (aprendizado!)

Antes: Entradas: apenas 0 ou 1

Agora: Entradas: valores reais

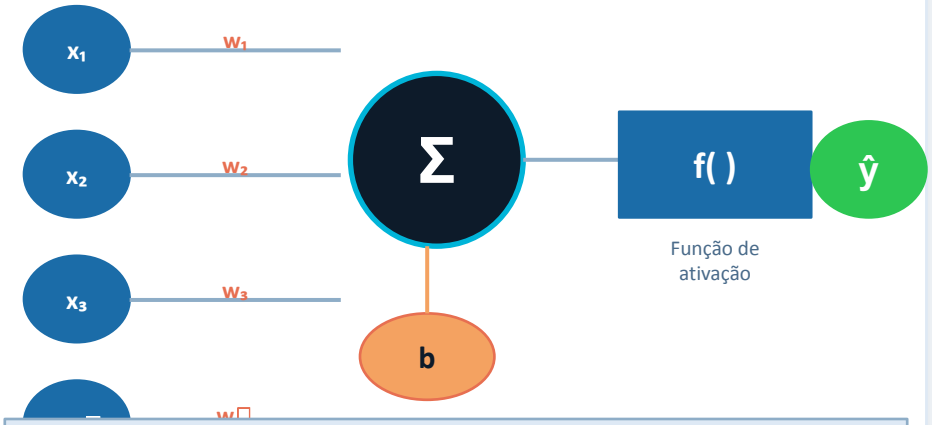
Antes: Limiar fixo θ

Agora: Bias b treinável

Antes: Sem algoritmo de treino

Agora: Regra delta: ajusta pesos pelo erro

Estrutura do Perceptron



Fórmula: $g(x) = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$

$\hat{y} = f(g(x)) \longrightarrow$ função de ativação (ex: degrau, sigmoid)

Inovação: os pesos w são atualizados automaticamente pelo algoritmo de aprendizado!

O Perceptron e a Separação Linear

O que é separação linear?

O Perceptron aprende uma reta (hiperplano) que separa duas classes nos dados.

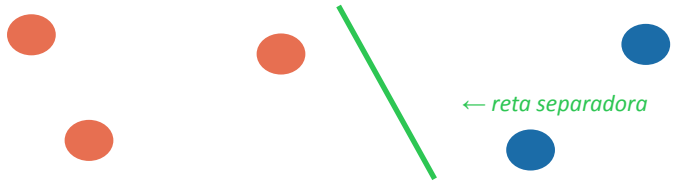
A equação da reta:

$$w_1x_1 + w_2x_2 + b = 0$$

Pontos acima da reta ⟶ classe 1
Pontos abaixo da reta ⟶ classe 0

Funciona bem quando os dados são linearmente separáveis!

AND — Separável! ✓ Perceptron consegue



XOR — NÃO separável! ✗ Perceptron falha

Impossível traçar uma única reta que separe os dois grupos!

1969 — O Problema do XOR

Minsky & Papert (1969) demonstraram matematicamente que o Perceptron simples é incapaz de aprender a função XOR.

XOR só retorna 1 quando as entradas são DIFERENTES — e isso não é linearmente separável.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

❄️ Inverno da IA (1969–1986)

O financiamento em pesquisa de redes neurais despencou. Comunidade científica questionou a viabilidade da abordagem. A solução exigia mais de uma camada — mas como treiná-la?

1986 — MLP e Backpropagation (Hinton)

O que é o MLP?

Multi-Layer Perceptron = rede neural com múltiplas camadas.

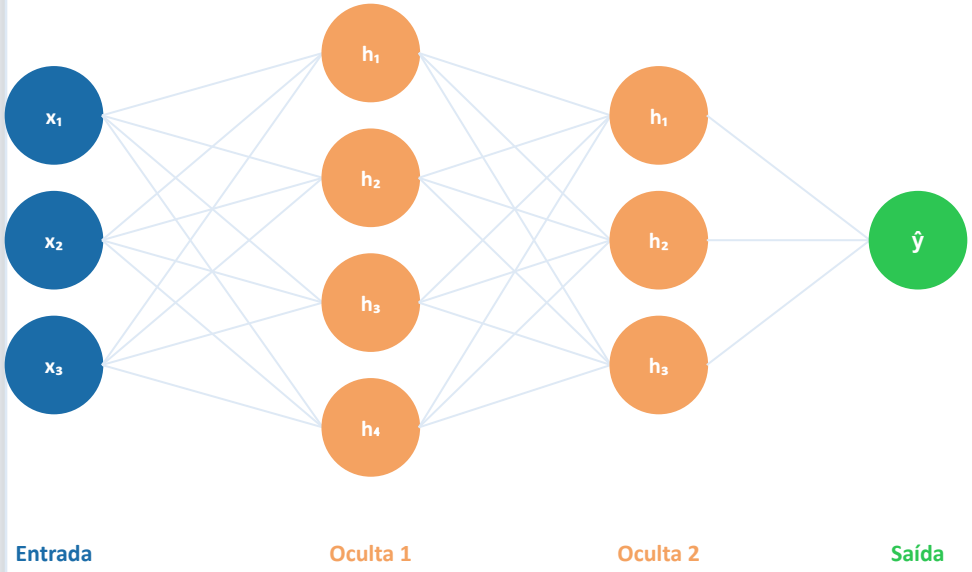
Camada de entrada (Input Layer):
Recebe os dados brutos (ex: pixels, valores)

Camadas ocultas (Hidden Layers):
Aprendem representações abstratas dos dados

Camada de saída (Output Layer):
Produz a predição final

O Backpropagation (1986) resolveu o problema de como ajustar os pesos de todas as camadas simultaneamente!

Arquitetura do MLP



◀ Backpropagation: erro propaga de trás para frente, ajustando todos os pesos

Funções de Ativação

Por que elas são essenciais?

Sem funções de ativação não-lineares, uma rede neural com N camadas seria equivalente a apenas 1 camada.

A não-linearidade permite que a rede aprenda padrões complexos no mundo real.

A seguir: Sigmoid, ReLU e Tanh — na prática com o notebook!

As 3 Funções de Ativação do Notebook

Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

Formato: S suave entre 0 e 1

Saída: entre 0 e 1

- Interpreta saída como probabilidade
- Vanishing gradient em redes profundas

Uso: Camada de saída para classificação binária

```
activation = nn.Sigmoid()
```

ReLU

$$f(x) = \max(0, x)$$

Formato: Zero para negativos, linear para positivos

Saída: 0 se x<0, senão x

- Rápida, evita vanishing gradient
- Neurônio pode 'morrer' (saída sempre 0)

Uso: Camadas ocultas — padrão atual em DL

```
activation = nn.ReLU()
```

Tanh

$$\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$$

Formato: S entre -1 e +1

Saída: entre -1 e +1

- Centrada em zero, melhor gradiente
- Vanishing gradient em valores extremos

Uso: RNNs e tarefas que precisam de sinal negativo

```
activation = nn.Tanh()
```

⬆️ *Cada linha de código PyTorch será demonstrada ao vivo no notebook!*

O Que o Notebook Demonstra?

1

Dados classificáveis

Cria um dataset 2D com sklearn (make_classification). Os pontos são plotados num plano com dois grupos — exatamente o problema que o Perceptron resolve.

```
X, Y = make_classification(n_features=2, ...)
```

2

Perceptron com pesos definidos manualmente

Constrói um Perceptron com PyTorch (nn.Linear) e define os pesos w1=5, w2=1 e bias=-0.4 à mão — reforçando o conceito da equação da reta.

```
perceptron.weight = nn.Parameter(torch.Tensor([[w1,
w2]]))
```

3

Comparação das 3 funções de ativação

Para 4 pontos específicos do dataset, calcula-se: o valor bruto g(x), a saída pelo limiar (0/1) e a saída pela função de ativação (Sigmoid, ReLU ou Tanh).

```
ret = perceptron(x);  act = activation(ret)
```

4

Visualização — o impacto da escolha

Cada função produz saídas diferentes para os mesmos pontos. O gráfico torna visível como a função de ativação afeta a interpretação do modelo.

```
activation = nn.Sigmoid() / nn.ReLU() / nn.Tanh()
```

Como Interpretar os Resultados do Notebook

Função	Valor bruto g(x)	Saída por limiar	Saída ativada	Interpretação
Sigmoid	3.6	1	0.97	Alta confiança — classe 1, probabilidade 97%
Sigmoid	-2.1	0	0.11	Baixa confiança — classe 0, probabilidade 89%
ReLU	3.6	1	3.6	Passa o valor original para a próxima camada
ReLU	-2.1	0	0.0	Neurônio silencioso — não propaga informação
Tanh	3.6	1	0.998	Fortemente positivo — próximo a +1
Tanh	-2.1	0	-0.97	Fortemente negativo — sinal útil para backprop

No notebook: execute cada célula, observe os valores impressos e o gráfico. Troque a função de ativação e compare os resultados!

Da Teoria à Prática — Onde Estamos?

1943

McCulloch & Pitts

Neurônio binário. Sem aprendizado.

1957

Perceptron

Pesos treináveis. 1 camada. Separação linear.

1986

MLP + Backprop

Múltiplas camadas. Aprende XOR e funções complexas.

▲ Você está aqui hoje

2006+

Deep Learning

Dezenas de camadas. Visão, linguagem, geração.

2020+

LLMs / ChatGPT

Bilhões de parâmetros. Conversação, código, arte.

Resumo da Aula

⚡ McCulloch & Pitts (1943): primeiro neurônio artificial — lógica binária, sem aprendizado

📏 Perceptron (1957): pesos treináveis, separação linear, limitado ao que é linearmente separável

❄️ Crise do XOR (1969): Minsky & Papert mostram o limite do Perceptron simples

🧠 MLP + Backpropagation (1986): múltiplas camadas, aprende qualquer função

⚙️ Funções de Ativação: Sigmoid, ReLU e Tanh introduzem a não-linearidade essencial

💻 Notebook: demonstração prática do impacto de cada função de ativação com PyTorch